Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt 1 - część druga
Klasyfikacja zbioru MNIST wielowarstwowym perceptronem

Przedmiot: Sieci Neuronowe

Autorzy:
Jakub Rak
Krzysztof Radzewicz

Prowadzący:
mgr inż. Mikołaj Małkiński

Warszawa, 3 listopada 2026

Spis treści

1 Opis zbioru danych MNIST **1**

2 Zmiany w kodzie **1**

3 Przebieg eksperymentu i analiza wyników **1**
3.1 Konfiguracja modelu i procedura uczenia 1
3.2 Dynamika uczenia — krzywe straty i dokładności 2
3.3 Ewaluacja na zbiorze testowym i macierz pomyłek 3

4 Krótkie podsumowanie i wnioski **5**

1 Opis zbioru danych MNIST

Niniejszy raport stanowi uzupełnienie pierwszego projektu laboratoryjnego. W pierwszej części model **MLP** testowany był na syntetycznych zbiorach dwuwymiarowych (klasyfikacja $\mathbb{R}^2 \to \{0, 1\}$ oraz $\mathbb{R}^2 \to \{0, 1, 2\}$) oraz regresji jednowymiarowej ($\mathbb{R} \to \mathbb{R}$). W tej części oceniono możliwość skalowania zaimplementowanej sieci neuronowej do danych o większym wymiarze wejściowym - klasycznym zbiorze *MNIST* w zadaniu klasyfikajci odręcznie pisanych cyfr.

Zbiór *MNIST* zawiera łącznie 70 000 znormalizowanych, czarno-białych (zapisanych w 256 poziomach szarości) obrazów przedstawiających cyfry arabskie od 0 do 9. Dane rozbito na dwa zestawy:

- **Oficjalny zbiór treningowy**: 60 000 obrazów, z których w eksperymencie 55 000 służyło do optymalizacji parametrów, a 5 000 do walidacji,
- **Zbiór testowy**: 10 000 niezależnych obrazów służących do ostatecznej ewaluacji sieci.

Każda próbka to obrazek o stałych wymiarach 28×28 pikseli. Każdy piksel reprezentowany jest przez liczbę całkowitą z zakresu $[0, 255]$, gdzie wartość 0 oznacza tło, a 255 maksymalną intensywność piksela.

W eksperymencie obrazy poddano spłaszczeniu do jednowymiarowego wektora cech:

$$\mathbf{x} \in \mathbb{R}^{784}, \quad D = 28 \times 28 = 784. \tag{1}$$

Wartości pikseli zostały znormalizowane do przedziału $[0, 1]$. Etykiety klas reprezentują 10 wzajemnie wykluczających się kategorii ($y \in \{0, 1, \ldots, 9\}$), które zakodowano w postaci wektorów *one-hot* o długości 10.

2 Zmiany w kodzie

Implementacja modelu **MLP** nie uległa żadnym modyfikacjom. Dodano jedynie dwa moduły pomocnicze:

- **Wczytywanie danych** (`experiments/common/data_loader.py`): funkcja odczytująca binarne pliki IDX zbioru MNIST, spłaszczająca obrazy 28×28 do wektorów o długości 784, normalizująca wartości pikseli do przedziału $[0, 1]$ oraz kodująca etykiety w formacie *one-hot*.
- **Wizualizacja wyników** (`experiments/common/plotting.py`): funkcje do wykreślania krzywych uczenia oraz generowania macierzy pomyłek wraz z metrykami ewaluacyjnymi na zbiorze testowym.

3 Przebieg eksperymentu i analiza wyników

3.1 Konfiguracja modelu i procedura uczenia

Eksperyment zrealizowano w skrypcie `experiments/run_mnist.py`. Krzywe dynamiki uczenia uśredniono po 10 uruchomieniach z ziarnami losowości `54217-54226`. Ziarna te sterowały inicjalizacją wag i tasowaniem mini-batchy. Podział na próbki uczące i walidacyjne był taki sam dla wszystkich uruchomień (ziarno podziału: `54217`), dzięki czemu pasma zmienności opisują wpływ losowości treningu, a nie zmiany składu podzbiorów. Tabela metryk, macierz pomyłek i przykłady błędów w dalszej części raportu dotyczą pojedynczego modelu z dotychczasowym ziarnem `54217`. Z oficjalnego zbioru treningowego (60 000 próbek) losowo wydzielono 5 000 próbek na podzbiór walidacyjny (służył do wyświetlania realnych postępów w uczeniu), a pozostałe 55 000 próbek przeznaczono do treningu. Niezależny zbiór testowy liczył 10 000 próbek. Liczebność

testu potwierdzono w nagłówkach obu lokalnych plików IDX (`t10k-images.idx3-ubyte` i `t10k-labels.idx1-ubyte`); ewaluacja korzystała z całego tego zbioru, bez opcji ograniczającej liczbę próbek.

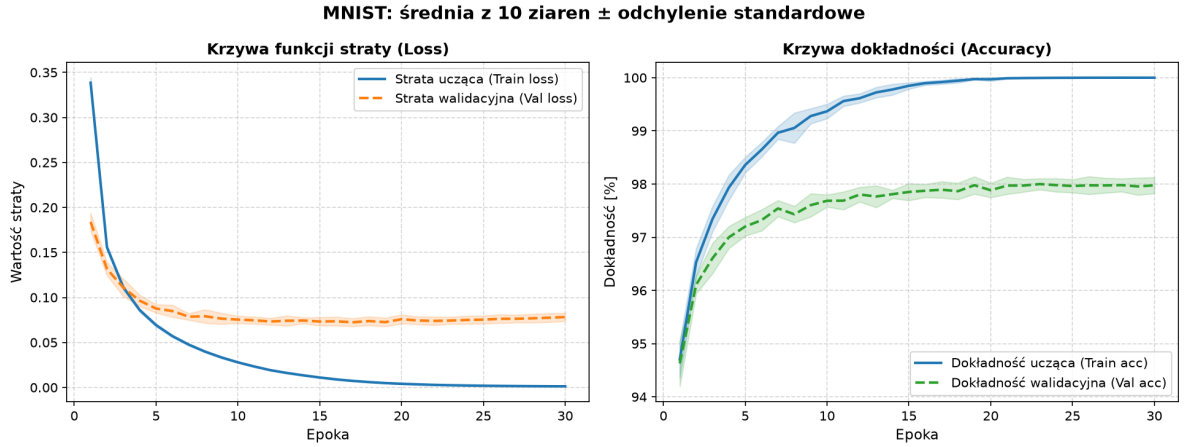
Wykorzystano następującą architekturę: $[784, 150, 50, 10]$.

- **Aktywacje ukryte**: wszystkie warstwy ukryte z aktywacją `ReLU` i inicjalizacją wag *He*,
- **Warstwa wyjściowa**: 10 neuronów z aktywacją `Softmax`, odpowiadających prawdopodobieństwom przynależności do klas $0, 1, \ldots, 9$.

We wszystkich warstwach włączono wektory *bias*. Jako funkcję kosztu zastosowano kategoryczną entropię krzyżową (`CategoricalCrossEntropyLoss`). Optymalizację prowadzono za pomocą klasycznego `SGD` ze stałym współczynnikiem uczenia $\eta = 0.05$. Sieć trenowano przez 30 epok z rozmiarem mini-batcha równym 32.

3.2 Dynamika uczenia — krzywe straty i dokładności

Na Rysunku 1 przedstawiono przebieg uczenia modelu na przestrzeni 30 epok. Każda linia przedstawia średnią danej metryki w danej epoce z 10 ziaren, a półprzezroczyste pasmo zakres średnia ± jedno odchylenie standardowe. Odchylenie obliczono jako $\sigma_e = \sqrt{\frac{1}{10}\sum_{s=1}^{10}(m_{s,e} - \bar{m}_e)^2}$, gdzie $m_{s,e}$ jest wartością metryki dla ziarna s w epoce e. Pasmo opisuje rozrzut wyników między uruchomieniami, nie przedział ufności średniej.



Rysunek 1: Przebieg uczenia modelu MLP $[784, 150, 50, 10]$ na zbiorze MNIST: strata kategorycznej entropii krzyżowej (panel lewy) oraz dokładność klasyfikacji w procentach (panel prawy) na zbiorze uczącym i walidacyjnym. Linie: średnie z 10 ziaren; pasma: $\pm 1\sigma$ między ziarnami.

Analiza krzywych uczenia pozwala sformułować następujące obserwacje:

- **Funkcja straty**: W lewym panelu widać gwałtowny spadek straty w pierwszych 5 epokach. Średnia strata treningowa zbliża się do zera. Stratę walidacyjną należy interpretować razem z pasmem zmienności: jej wzrost mimo dalszego spadku straty treningowej jest oznaką przeuczenia według tej metryki, nawet jeśli dokładność walidacyjna pozostaje wysoka. Minimum średniej straty walidacyjnej przypada na 17. epokę; w 30. epoce wynosi ona 0.0782 ± 0.0045, przy stracie treningowej około 0.00124.
- **Krzywa dokładności**: Dokładność rośnie szybko już w pierwszych epokach. Jest tak ponieważ wagi są aktualizowane co mini batch, co w połączeniu z bardzo dużym zbiorem treningowym (55 000 próbek) i rozmiarem mini batch 32, daje bardzo wiele aktualizacji.

Pod koniec treningu dokładność ucząca jest bliska 100%. W 30. epoce średnia dokładność walidacyjna wynosi 97.976%, a jej odchylenie standardowe między ziarnami to 0.150 punktu procentowego.

- **Zmienność między ziarnami**: Pasma pokazują wrażliwość procesu uczenia na inicjalizację wag i kolejność mini-batchy przy stałym podziale danych. Pozwala to ocenić powtarzalność przebiegu uczenia bez opierania wniosków wyłącznie na pojedynczym uruchomieniu.

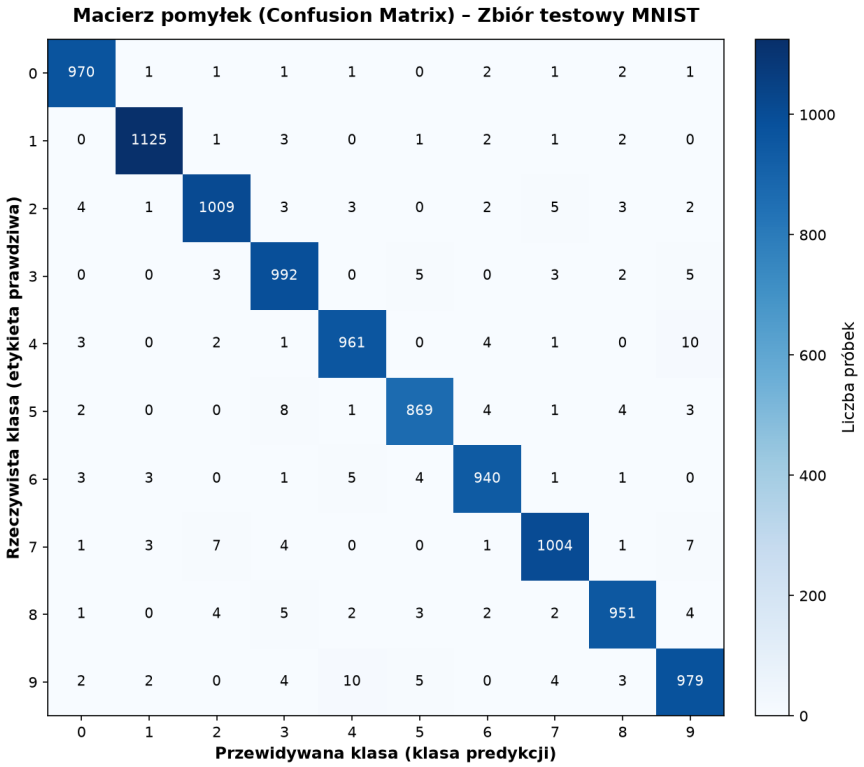
3.3 Ewaluacja na zbiorze testowym i macierz pomyłek

Ostateczna ewaluacja modelu dla ziarna `54217` na 10 000 niezależnych próbkach testowych wykazała całkowitą dokładność na poziomie **98.00%** (9 800 poprawnie sklasyfikowanych obrazów na 10 000). Szczegółowe metryki w rozbiciu na poszczególne cyfry zebrano w Tabeli 1.

Tabela 1: Szczegółowe metryki jakości klasyfikacji modelu MLP na zbiorze testowym MNIST ($N = 10\ 000$).

Klasa	**Liczba wystąpień**	**Recall (TPR)**	**Precision (PPV)**	**Specificity (TNR)**	**FPR**	F_1**-Score**
0	980	0.9898	0.9838	0.9982	0.0018	0.9868
1	1135	0.9912	0.9912	0.9989	0.0011	0.9912
2	1032	0.9777	0.9825	0.9980	0.0020	0.9801
3	1010	0.9822	0.9706	0.9967	0.0033	0.9764
4	982	0.9786	0.9776	0.9976	0.0024	0.9781
5	892	0.9742	0.9797	0.9980	0.0020	0.9770
6	958	0.9812	0.9822	0.9981	0.0019	0.9817
7	1028	0.9767	0.9814	0.9979	0.0021	0.9790
8	974	0.9764	0.9814	0.9980	0.0020	0.9789
9	1009	0.9703	0.9683	0.9964	0.0036	0.9693
Średnia arytmetyczna (makro)		0.9798	0.9799	0.9978	0.0022	0.9798
Średnia ważona ($N = 10\ 000$)		0.9800	0.9800	0.9978	0.0022	0.9800

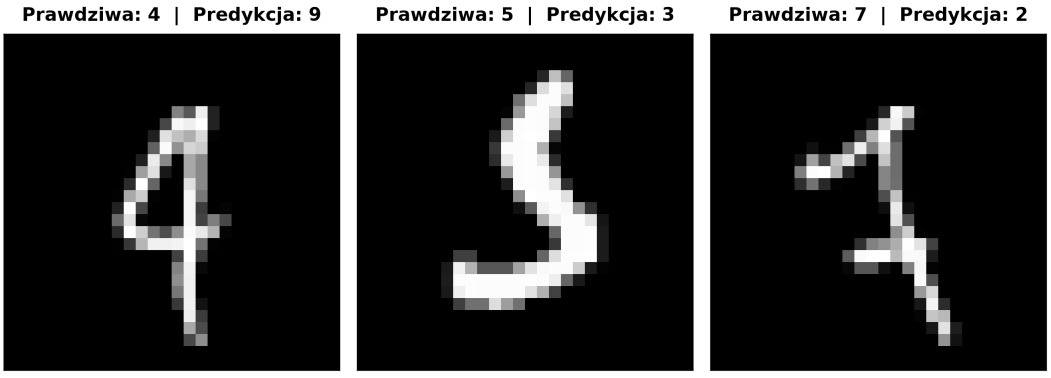
Strukturę błędów popełnianych przez sieć szczegółowo ilustruje macierz pomyłek zaprezentowana na Rysunku 2.



Rysunek 2: Znormalizowana macierz pomyłek na zbiorze testowym MNIST ($N = 10\,000$). Wiersze odpowiadają etykietom prawdziwym, natomiast kolumny predykcjom modelu.

Z analizy macierzy pomyłek i tabeli metryk płyną konkretne wnioski:

- **Najłatwiejsze klasy**: Model najskuteczniej rozpoznaje cyfrę *1* ($F_1 = 0.9912$, 1125 trafień na 1135 próbek) oraz cyfrę *0* ($F_1 = 0.9868$, 970 trafień na 980 próbek). Obie te cyfry są bardzo proste geometrycznie, co prawdopodobnie redukuje liczbę pomyłek.
- **Przykłady błędnie sklasyfikowanych próbek**: Rysunek 3 przedstawia trzy przykładowe próbki ze zbioru testowego, dla których model podjął błędną decyzję: cyfrę 4 sklasyfikowaną jako 9, cyfrę 5 jako 3 oraz cyfrę 7 jako 2.



Rysunek 3: Przykładowe obrazy ze zbioru testowego MNIST błędnie sklasyfikowane przez model MLP, obrazujące główne pomyłki modelu.

- **Bardzo niska liczba fałszywych alarmów**: Wartości *TNR* dla wszystkich klas wynoszą co najmniej 0.9964, a odsetek fałszywych alarmów *FPR* nie przekracza 0.36%, co wskazuje na niezawodność aktywacji Softmax bez faworyzowania żadnej z kategorii.

4 Krótkie podsumowanie i wnioski

Przeprowadzony eksperyment na zbiorze MNIST pozwolił zweryfikować zachowanie zaimplementowanego wielowarstwowego perceptronu w znacznie bardziej wymagających warunkach niż proste dane dwuwymiarowe. Główne wnioski można podsumować następująco:

1. **Poprawność i skalowalność autorskiej implementacji**: Biblioteka `py_nn1` bez żadnych modyfikacji poradziła sobie z optymalizacją modelu posiadającego ponad 125 000 parametrów wagowych ($784 \times 150 + 150 \times 50 + 50 \times 10 \approx 125\,850$ wag wraz z biasami) na zbiorze 55 000 próbek uczących. Obliczenia przebiegły stabilnie, bez problemów zanikających czy eksplodujących gradientów.

2. **Wysoka skuteczność klasycznego MLP**: Standardowy perceptron z dwiema warstwami ukrytymi (150 i 50 neuronów), nieliniowością `ReLU` i inicjalizacją *He* uzyskał **98.00% dokładności** na 10 000 obrazów testowych.

3. **Sprawność optymalizatora SGD**: Prosty algorytm stochastycznego spadku gradientu z $\eta = 0.05$ i wielkością mini batcha 32 okazał się w pełni wystarczający do osiągnięcia zbieżności w granicach 15–20 epok, zapewniając jednocześnie odpowiednią jakość predykcji.

4. **Błędy klasyfikacji**: Można przypuszczać że spośród nielicznych 200 błędów (na 10 000 przypadków testowych) istotna część z nich wynika z podobieństwa geometrycznego między klasami cyfr. Warto byłoby zbadać czy takie błędy można ograniczyć stosując model który zamiast spłaszczonego wektora, przyjmowałby wejście zachowujące dwuwymiarowe zależności (na przykład konwolucyjną sieć neuronową).